Offensive Security, Saint Cyber

exercise booklet

Henrik Kramselund

hlk@zencurity.com

November 1, 2026



Note: exercises marked with ⚠ are considered important. These contain subjects that are essential for the course. Even if you don't work through the exercise, you might want to know the subjects covered by these.

Exercises marked with ℹ are considered optional. These contain subjects that are related to the course, but less important. You may want to browse these and if interested work through them. They may require more time than we have available during the course.

Contents

I Introduction & Reconnaissance 4

1 Define scope and rules of engagement 5

2 Score a vulnerability with CVSS 6

3 Lookup Whois and DNS data 7

4 Capturing network packets 8

5 Discover active systems ping sweep 9

6 Execute nmap TCP and UDP port scan 10

7 Perform nmap OS detection 11

8 Perform nmap service scan 12

9 Nmap full scan 13

10 Nping check ports 14

11 Nmap Scripting Engine NSE scripts 16

12 Try Nmap from Metasploit 18

13 Reporting HTML 19

14 Bonus: Try masscan 21

II Network Attacks 22

15 Sniffing network packets 23

16 Discovery using ping and traceroute 24

17 ICMP tool - icmpush 25

© license CC BY 3.0. 2026 Henrik Kramselund

18 Research arpspoof and dsniff 26

19 Find systems with SNMP 27

20 Bonus:TCP SYN flooding 30min 28

21 Scapy packet crafting 30

22 Try Hydra brute force 31

III Web Attacks 32

23 ⚠ Run OWASP Juice Shop 45 min 33

24 ⚠ Setup JuiceShop environment, app and proxy - up to 60min 35

25 ⚠ JuiceShop Attacks 60min 37

26 ⚠ SSL/TLS scanners 15 min 39

27 ⚠ Internet scanners 15 min 40

28 ⚠ Nikto Web Scanner 15 min 41

29 ⚠ Whatweb Scanner 15 min 43

30 Real Vulnerabilities up to 30min 44

31 ⚠ PHP Passwords 30 min 45

32 Apache Benchmark 20 min 46

33 ⚠ Trying PMD static analysis 30 min 48

34 ⚠ Git hook 30 min 49

© license CC BY 3.0. 2026 Henrik Kramselund

Preface

This exercise booklet accompanies the Offensive Security subject. Exercises are reused from the instructor's own course material (nmap-workshop, hackerworkshop and security-in-web-development). There are deliberately a few more exercises than needed, so groups that have already done some (for example Nmap) still have fresh material. FIXME marks exercises still to be written.

© license CC BY 3.0. 2026 Henrik Kramselund

Part I

Introduction & Reconnaissance

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 1

Define scope and rules of engagement

- **FIXME:** write a scope + rules-of-engagement exercise for a given target (obj 4)

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 2

Score a vulnerability with CVSS

- **FIXME:** pick a current CVE; work the base vector and score (obj 6)

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 3

Lookup Whois and DNS data

Objective:
Learn to use DNS and Whois databases - lookup the IP address of your current connection. The IP of the main web server of www.zencurity.com, the mail server for zencurity.com

Purpose:
Knowing who to contact in case of problems on the internet is important, and also verifying before starting scanning is required.

Suggested method:
Use the website of RIPE NCC `https://www.ripe.net/` or their other site

`https://stat.ripe.net/`

Use command line tools host and dig on Kali Linux.

```
host www.zencurity.com
host -t mx zencurity.com
```

Hints:
Whois databases are distributed to Regional Internet Registries such as ARIN, AfriNIC, RIPE, LACNIC and APNIC.

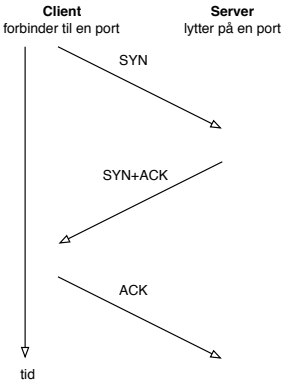
Solution:
If you are using Linux or Mac you have a command line tool too:
Use the command whois with an IP address, `whois 185.129.60.130`.

Discussion:
The whois system was implemented after the Morris Worm affected the internet in November 1988, because it was realized that the internet had grown to a size that required more management.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 4

Capturing network packets



Objective:
Sniff packets and dissect them using Wireshark

Purpose:
See real network traffic, also know that a lot of information is available and not encrypted.

Note the three way handshake between hosts

Suggested method:
Open Wireshark and start a capture
Then in another window execute the ping program while sniffing

or perform a Telnet connection while capturing data

Hints:
When running on Linux the network cards are usually named eth0 for the first Ethernet and wlan0 for the first Wireless network card. In Windows the names of the network cards are long and if you cannot see which cards to use then try them one by one.

Solution:
When you have collected some packets you are done.

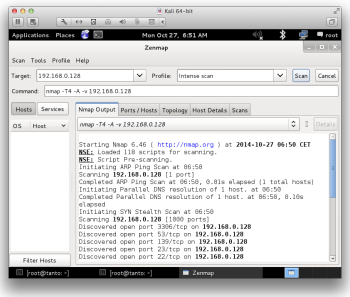
Discussion: Is it ethical to collect packets from an open wireless network?

Also note the TTL values in packets from different operating systems

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 5

Discover active systems ping sweep



Objective:
Use nmap to discover active systems

Purpose:
Know how to use nmap to scan networks for active systems.

Suggested method:
Try different scans,

- Ping sweep to find active systems
- Port sweeps to find active systems with specific ports

Hints:
Try nmap in sweep mode - and you may run this from Zenmap

Solution:
Use the command below as examples:

- Ping sweep `nmap -sP 10.0.45.*`
- Port sweeps `nmap -p 80 10.0.45.*`

Discussion:
Quick scans quickly reveal interesting hosts, ports and services

Also now make sure you understand difference between single host scan 10.0.45.123/32, a whole subnet /24 250 hosts 10.0.45.0/24 and other more advanced targeteting like 10.0.45.0/25 and 10.0.45.1-10

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 6

Execute nmap TCP and UDP port scan

Objective:
Use nmap to discover important open ports on active systems

Purpose:
Finding open ports will allow you to find vulnerabilities on these ports.

Suggested method:
Use `nmap -p 1-1024 server` to scan the first 1024 TCP ports and use Nmap without ports. What is scanned then?

Try to use `nmap -sU` to scan using UDP ports, not really possible if a firewall is in place.

If a firewall blocks ICMP you might need to add `-P0` or even `-PN` to make nmap scan even if there are no Ping responses

Hints:
Sample command: `nmap -P0 -sU -p1-1024 server` UDP port scanning 1024 ports without doing a Ping first

Solution:
Discover some active systems and most interesting ports, which are 1-1024 and the built-in list of popular ports.

Discussion:
There is a lot of documentation about the nmap portscanner, even a book by the author of nmap. Make sure to visit http://www.nmap.org

TCP and UDP is very different when scanning. TCP is connection/flow oriented and requires a handshake which is very easy to identify. UDP does not have a handshake and most applications will not respond to probes from nmap. If there is no firewall the operating system will respond to UDP probes on closed ports - and the ones that do not respond must be open.

When doing UDP scan on the internet you will almost never get a response, so you cannot tell open (not responding services) from blocked ports (firewall drop packets). Instead try using specific service programs for the services, sample program could be `nsping` which sends DNS packets, and will often get a response from a DNS server running on UDP port 53.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 7

Perform nmap OS detection

Objective:
Use nmap OS detection and see if you can guess the brand of devices on the network

Purpose:
Getting the operating system of a system will allow you to focus your next attacks.

Suggested method:
Look at the list of active systems, or do a ping sweep.

Then add the OS detection using the option `-O`

Better to use -A all the time, includes even more scripts and advanced stuff See the next exercise.

Hints:

The nmap can send a lot of packets that will get different responses, depending on the operating system. TCP/IP is implemented using various constants chosen by the implementors, they have chosen different standard packet TTL etc.

Solution:
Use a command like `nmap -O -p1-100 10.0.45.45` or `nmap -A -p1-100 10.0.45.45`

Discussion:
nmap OS detection is not a full proof way of knowing the actual operating system, but in most cases in can detect the family and in some cases it can identify the exact patch level of the system.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 8

Perform nmap service scan

Objective:
Use more advanced features in Nmap to discover services.

Purpose:
Getting more intimate with the system will allow more precise discovery of the vulnerabilities and also allow you to select the next tools to run.

Suggested method:
Use `nmap -A` option for enabling service detection and scripts

Hints:
Look into the manual page of nmap or the web site book about nmap scanning

Solution:
Run nmap and get results.

Discussion:

Some services will show software versions allowing an attacker easy lookup at web sites to known vulnerabilities and often exploits that will have a high probability of success.

Make sure you know the difference between a vulnerability which is discovered, but not really there, a false positive, and a vulnerability not found due to limitations in the testing tool/method, a false negative.

A sample false positive might be reporting that a Windows server has a vulnerability that you know only to exist in Unix systems.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 9

Nmap full scan

Objective:
Write down your Nmap strategy, and if needed create your own Nmap profile in Zenmap.

Purpose:
Doing a port scan often requires you to run multipl Nmap scans.

Suggested method:
Use Zenmap to do:

1. A few quick scans, to get web servers and start web scanners/crawlers
2. Full scan of all TCP ports, -p 1-65535
3. Full or limited UDP scan, `nmap -sU --top-ports 100`
4. Specialized scans, like specific source ports

Hints:
Using a specific source ports using -g/–source-port <portnum>: Use given port number with ports like FTP 20, DNS 53 can sometimes get around router filters and other stateless Access Control Lists

Solution:
Run nmap and get results.

Discussion:
Recommendation it is highly recommended to always use:

```
-iL <inputfilename>: Input from list of hosts/networks
-oA outputbasename: output in all formats, see later
```

Some examples of real life Nmaps I have run recently:

```
dns-scan: nmap -sU -p 53 --script=dns-recursion -iL targets -oA dns-recursive
bgpscan: nmap -A -p 179 -oA bgpscan -iL targets
dns-recursive: nmap -sU -p 53 --script=dns-recursion -iL targets -oA dns-recursive
php-scan: nmap -sV --script=http-php-version -p80,443 -oA php-scan -iL targets
scan-vtep-tcp: nmap -A -p 1-65535 -oA scan-vtep-tcp 10.1.2.3 192.0.2.123
snmp-10.x.y.0.gnmap: nmap -sV -A -p 161 -sU --script=snmp-info -oA snmp-10xy 10.x.
snmpscan: nmap -sU -p 161 -oA snmpscan --script=snmp-interfaces -iL targets
sshscan: nmap -A -p 22 -oA sshscan -iL targets
vncscan: nmap -A -p 5900-5905 -oA vncscan -iL targets
```

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 10

Nping check ports

Objective:
Show the use of Nping tool for checking ports through a network

Purpose:
Nping can check if probes can reach through a network, reporting success of failure. Allows very specific packets to be sent.

Suggested method:

```
root@KaliVM:~# nping --tcp -p 80 www.zencurity.com

Starting Nping 0.7.70 ( https://nmap.org/nping ) at 2018-09-07 19:06 CEST
SENT (0.0300s) TCP 10.137.0.24:3805 > 185.129.60.130:80 S ttl=64 id=18933 iplen=40  seq=2984847972
RCVD (0.0353s) TCP 185.129.60.130:80 > 10.137.0.24:3805 SA ttl=56 id=49674 iplen=44  seq=365459769
SENT (1.0305s) TCP 10.137.0.24:3805 > 185.129.60.130:80 S ttl=64 id=18933 iplen=40  seq=2984847972
RCVD (1.0391s) TCP 185.129.60.130:80 > 10.137.0.24:3805 SA ttl=56 id=50237 iplen=44  seq=234792649
SENT (2.0325s) TCP 10.137.0.24:3805 > 185.129.60.130:80 S ttl=64 id=18933 iplen=40  seq=2984847972
RCVD (2.0724s) TCP 185.129.60.130:80 > 10.137.0.24:3805 SA ttl=56 id=9842 iplen=44  seq=2355974413
SENT (3.0340s) TCP 10.137.0.24:3805 > 185.129.60.130:80 S ttl=64 id=18933 iplen=40  seq=2984847972
RCVD (3.0387s) TCP 185.129.60.130:80 > 10.137.0.24:3805 SA ttl=56 id=1836 iplen=44  seq=3230085295
SENT (4.0362s) TCP 10.137.0.24:3805 > 185.129.60.130:80 S ttl=64 id=18933 iplen=40  seq=2984847972
RCVD (4.0549s) TCP 185.129.60.130:80 > 10.137.0.24:3805 SA ttl=56 id=62226 iplen=44  seq=303349222

Max rtt: 40.044ms | Min rtt: 4.677ms | Avg rtt: 15.398ms
Raw packets sent: 5 (200B) | Rcvd: 5 (220B) | Lost: 0 (0.00%)
Nping done: 1 IP address pinged in 4.07 seconds
```

Hints:
A lot of options are similar to Nmap

Solution:

Discussion:
A colleague of ours had problems sending specific IPsec packets through a provider. Using a tool like Nping it is possible to show what happens, or where things are blocked.

Things like changing the TTL may provoke ICMP messages, like this:

```
root@KaliVM:~# nping --tcp -p 80 --ttl 3 www.zencurity.com

Starting Nping 0.7.70 ( https://nmap.org/nping ) at 2018-09-07 19:08 CEST
SENT (0.0303s) TCP 10.137.0.24:37244 > 185.129.60.130:80 S ttl=3 id=60780 iplen=40  seq=1997801125 w
RCVD (0.0331s) ICMP [10.50.43.225 > 10.137.0.24 TTL=0 during transit (type=11/code=0) ] IP [ttl=62 i
SENT (1.0314s) TCP 10.137.0.24:37244 > 185.129.60.130:80 S ttl=3 id=60780 iplen=40  seq=1997801125 w
RCVD (1.0337s) ICMP [10.50.43.225 > 10.137.0.24 TTL=0 during transit (type=11/code=0) ] IP [ttl=62 i
SENT (2.0330s) TCP 10.137.0.24:37244 > 185.129.60.130:80 S ttl=3 id=60780 iplen=40  seq=1997801125 w
RCVD (2.0364s) ICMP [10.50.43.225 > 10.137.0.24 TTL=0 during transit (type=11/code=0) ] IP [ttl=62 i
SENT (3.0346s) TCP 10.137.0.24:37244 > 185.129.60.130:80 S ttl=3 id=60780 iplen=40  seq=1997801125 w
RCVD (3.0733s) ICMP [10.50.43.225 > 10.137.0.24 TTL=0 during transit (type=11/code=0) ] IP [ttl=62 i
SENT (4.0366s) TCP 10.137.0.24:37244 > 185.129.60.130:80 S ttl=3 id=60780 iplen=40  seq=1997801125 w
RCVD (4.0558s) ICMP [10.50.43.225 > 10.137.0.24 TTL=0 during transit (type=11/code=0) ] IP [ttl=62 i
```

© license CC BY 3.0. 2026 Henrik Kramselund

```
Max rtt: 38.574ms | Min rtt: 2.248ms | Avg rtt: 13.143ms
Raw packets sent: 5 (200B) | Rcvd: 5 (360B) | Lost: 0 (0.00%)
Nping done: 1 IP address pinged in 4.07 seconds
```

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 11

Nmap Scripting Engine NSE scripts

Objective:
Show the use of NSE scripts, copy/modify a script written in Lua.

Purpose:
Investigate the scripts from Nmap, copy one, learn how to run specific script using options

Suggested method:

```
# cd /usr/share/nmap/scripts
# nmap --script http-default-accounts.nse www.zencurity.com
# cp http-default-accounts.nse http-default-accounts2.nse
# nmap --script http-default-accounts2.nse www.zencurity.com
Starting Nmap 7.70 ( https://nmap.org ) at 2018-09-07 19:45 CEST
...
```

This will allow you to make changes to existing scripts.

Hints:
We will do this quick and dirty - later when doing this at home, I recommend putting your scripts in your home directory or a common file hierarchy.

Solution:
Other examples

```
nmap --script http-enum 10.0.45.0/24
nmap -p 445 --script smb-os-discovery 10.0.45.0/24
```

Discussion:
There are often new scripts when new vulnerabilities are published. It is important to learn how to incorporate them into your scanning. When heartbleed roamed I was able to scan about 20.000 IPs for Heartbleed in less than 10 minutes, which enabled us to update our network quickly for this vulnerability.

It is also possible to run categories of scripts:

```
nmap --script "http-*"

     nmap --script "default or safe"
      This is functionally equivalent to nmap --script "default,safe". It loads al

     nmap --script "default and safe"
      Loads those scripts that are in both the default and safe categories.
```

or get help for a script:

```
# nmap -script-help http-vuln-cve2013-0156.nse
```

```
Starting Nmap 7.70 ( https://nmap.org ) at 2018-09-07 19:00 CEST

http-vuln-cve2013-0156
Categories: exploit vuln
https://nmap.org/nsedoc/scripts/http-vuln-cve2013-0156.html
  Detects Ruby on Rails servers vulnerable to object injection, remote command
  executions and denial of service attacks. (CVE-2013-0156)

  All Ruby on Rails versions before 2.3.15, 3.0.x before 3.0.19, 3.1.x before
  3.1.10, and 3.2.x before 3.2.11 are vulnerable. This script sends 3 harmless
  YAML payloads to detect vulnerable installations. If the malformed object
  receives a status 500 response, the server is processing YAML objects and
  therefore is likely vulnerable.

  References:
  * https://community.rapid7.com/community/metasploit/blog/2013/01/10/exploiting-ru
  * https://groups.google.com/forum/?fromgroups=#!msg/rubyonrails-security/61bkgvnS
  * http://cvedetails.com/cve/2013-0156/
```

Some scripts also require, or allow arguments into them:

```
  nmap -sC --script-args 'user=foo,pass=",=bar",paths=/admin,/cgi-bin,xmpp-info.ser
```

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 12

Try Nmap from Metasploit

All the things we did from Zenmap and Nmap command line can be done from inside Metasploit, or imported into Metasploit.

Try starting Metasploit from either Armitage, or command line:

```
root@cornerstone03:~# msfconsole
...

       =[ metasploit v4.17.1-dev                          ]
+ -- --=[ 1788 exploits - 1018 auxiliary - 310 post       ]
+ -- --=[ 538 payloads - 41 encoders - 10 nops            ]
+ -- --=[ Free Metasploit Pro trial: http://r-7.co/trymsp ]

msf >
msf > workspace -a demo1
[*] Added workspace: demo1
msf > db_nmap -p 80,443 www.zencurity.com
[*] Nmap: Starting Nmap 7.01 ( https://nmap.org ) at 2018-09-07 19:11 CEST
[*] Nmap: Nmap scan report for www.zencurity.com (185.129.60.130)
[*] Nmap: Host is up (0.00021s latency).
[*] Nmap: Other addresses for www.zencurity.com (not scanned): 2a06:d380:0:3065::80
[*] Nmap: PORT    STATE SERVICE
[*] Nmap: 80/tcp  open  http
[*] Nmap: 443/tcp open  https
[*] Nmap: Nmap done: 1 IP address (1 host up) scanned in 0.25 seconds
msf > hosts

Hosts
=====

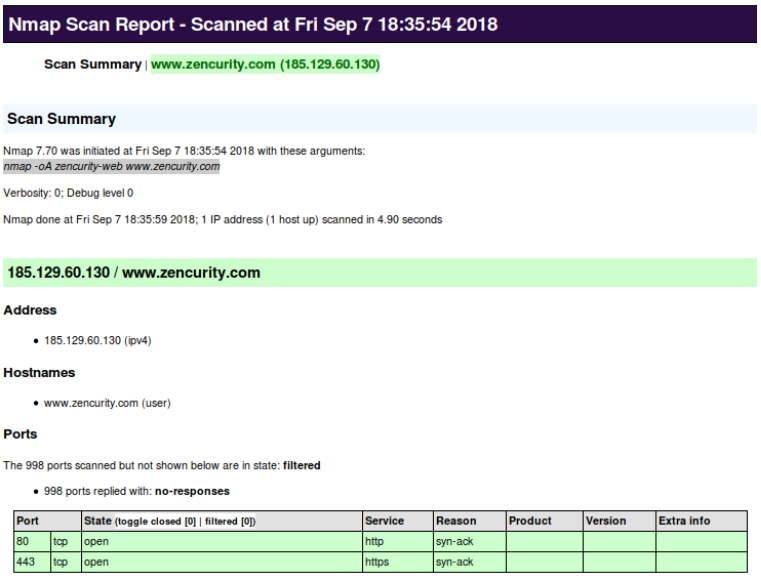
address         mac  name  os_name  os_flavor  os_sp  purpose  info  comments
-------         ---  ----  -------  ---------  -----  -------  ----  --------
185.129.60.130            Unknown                    device

msf > services
Services
========

host            port  proto  name   state  info
----            ----  -----  ----   -----  ----
185.129.60.130  80    tcp    http   open
185.129.60.130  443   tcp    https  open
```

Exercise 13

Reporting HTML



Nmap Scan Report - Scanned at Fri Sep 7 18:35:54 2018

Scan Summary | www.zencurity.com (185.129.60.130)

Scan Summary

Nmap 7.70 was initiated at Fri Sep 7 18:35:54 2018 with these arguments:
nmap -oA zencurity-web www.zencurity.com

Verbosity: 0; Debug level 0

Nmap done at Fri Sep 7 18:35:59 2018; 1 IP address (1 host up) scanned in 4.90 seconds

185.129.60.130 / www.zencurity.com

Address

- 185.129.60.130 (ipv4)

Hostnames

- www.zencurity.com (user)

Ports

The 998 ports scanned but not shown below are in state: filtered

- 998 ports replied with: no-responses

Port		State (toggle closed [0] | filtered [0])	Service	Reason	Product	Version	Extra info
80	tcp	open	http	syn-ack			
443	tcp	open	https	syn-ack			

Objective:
Show the use of XML output and convert to HTML

Purpose:
Reporting data is very important. Using the oA option Nmap can export data in three formats easily, each have their use. They are normal, XML, and grepable formats at once.

Suggested method:

```
sudo nmap -oA zencurity-web www.zencurity.com
xsltproc zencurity-web.xml > zencurity-web.html
```

Hints:
Nmap includes the stylesheet in XML and makes it very easy to create HTML.

Solution:
Run XML through xsltproc, command line XSLT processor, or another tool

Discussion:

Options you can use to change defaults:

```
--stylesheet <path/URL>: XSL stylesheet to transform XML output to HTML
--webxml: Reference stylesheet from Nmap.Org for more portable XML
```

Also check out the Ndiff tool

© license CC BY 3.0. 2026 Henrik Kramselund

```
hlk@cornerstone03:~$ ndiff zencurity-web.xml zencurity-web-2.xml
-Nmap 7.70 scan initiated Fri Sep 07 18:35:54 2018 as: nmap -oA zencurity-web www
+Nmap 7.70 scan initiated Fri Sep 07 18:46:01 2018 as: nmap -oA zencurity-web-2 w

 www.zencurity.com (185.129.60.130):
 PORT    STATE SERVICE VERSION
+443/tcp open  https
```

(I ran a scan, removed a port from the first XML file and re-scanned)

Exercise 14

Bonus: Try masscan

https://github.com/robertdavidgraham/masscan

This is the fastest Internet port scanner. It can scan the entire Internet in under 6 minutes, transmitting 10 million packets per second.

What is the difference between masscan and Nmap? Stateful vs stateless scan

© license CC BY 3.0. 2026 Henrik Kramselund

Part II

Network Attacks

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 15

Sniffing network packets

Objective:
Sniff packets and dissect them using Wireshark

Purpose:
See real network traffic, also know that a lot of information is available and not encrypted.

Suggested method:
Open Wireshark and start a capture - either from Windows or BackTrack
Then in another window execute the ping program while sniffing

Hints:
When running on Linux the network cards are named eth0 for the first Ethernet and wlan0 for the first Wireless network card. In Windows the names of the network cards are long and if you cannot see which cards to use then try them one by one.

Solution:
When you have collected some packets you are done.

Discussion: Is it ethical to collect packets from an open wireless network?

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 16

Discovery using ping and traceroute

Objective:
Learn how to use the ping and traceroute programs.

Purpose:
Doing network discovery is an important part of doing security testing.

Suggested method:
Use `ping` and `traceroute` testing your network connection.

Can be performed from both Windows and Unix/Linux

Remember though that traceroute is named `tracert` on Windows.

Hints:
ICMP is the Internet Control Message Protocol which is used for reporting problems back to a source on the internet. It can also be used for diagnosing problems using ICMP ECHO request packets. ICMP is very important when doing security testing for network discovery and making sure connections are alive.

The following protocols are being used:

- Ping uses ICMP packets with request and expect responses
- Tracert on Windows uses ICMP packets
- Traceroute on Unix by default uses UDP packets, but can also use ICMP

Solution:
Run the commands - not all are available on Windows, so perhaps use Unix:

- **traceroute** (Unix) or **tracert** (Windows)
- **traceroute -I**

Discussion: A lot of people just try to block any ICMP, but that will actually hurt a lot of functionality within your network.

Other trace programs exist, for example TCP traceroute programs - find them on the BackTrack!

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 17

ICMP tool - icmpush

Objective:
See a sample program that allows you to send ICMP packets without doing actual programming

Purpose:
Know that a lot of hacker programs exist on any level of IP

Suggested method:
Login to the Unix server - see the manual and use timestamp request packets

Alternative install icmpush on BackTrack using the command apt-get, try running icmpush and then follow on screen instructions.

Hints:

Solution:
Use the command `icmpush -v -tstamp 10.0.45.45` and also try echo, mask from the icmpush program

Discussion:
Other toolboxes for creating network packets are:

- Nemesis - which is on the BackTrack
- Scapy - which allow you to do Python programs that can send packets
- Hping - which is on the BackTrack

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 18

Research arpspoof and dsniff

Objective:
Read about arpspoof and dsniff

Purpose:
Realize that having a switch does not prevent sniffing, but makes it a bit more difficult.

Suggested method:
Log onto the Unix server and look at manual pages

Hints:
ARP spoofing is about sending false information to systems trying to communicate. If it happens the systems will send their packets to the wrong destination, the hacker who can then sniff data and forward.

Dsniff is a program that can decode a lot of older protocols.

Solution:
To read manual pages use: `man arpspoof` and `man dsniff`

Discussion:
What can be done using these programs?

Please notice that it can make the network a bit unstable if you are not carefull. Luckily the network will recover by itself in 5-10 minutes.

A graphical tool is available on the BackTrack named Ettercap.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 19

Find systems with SNMP

Objective:
Use snmpwalk to research SNMP systems

Purpose:
Learn that gathering information can help an attacker.

Suggested method:
Log into the Unix server provided and run snmpwalk which is using UDP port 161.

Hints:
We are running in a LAN environment with less firewalls, so doing nmap UDP scan is possible.

When discovering an IP then use the `snmpwalk` program to show a lot of information.

Solution:

- Use the command `snmpwalk -v 2c -c public 10.0.45.34 | less`

The command less will show output one screen at a time.

Discussion:
In real networks SNMP is being used a lot, but new equipment is starting NOT to allow access using the community string public.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 20

Bonus:TCP SYN flooding 30min

Objective:
Start a webserver attack using SYN flooding tool hping3.

Purpose:
See how easy it is to produce packets on a network using hacker programs.

The tool we will use is very flexible and can produce ICMP, UDP and TCP using very few options. This tool is my primary one for doing professsional DDoS testing.

```
-1 --icmp
       ICMP  mode,  by  default  hping3  will  send  ICMP echo-request, you can set other ICMP
       type/code using --icmptype --icmpcode options.

-2 --udp
       UDP mode, by default hping3 will send udp to target host's port 0.  UDP header  tunable
       options are the following: --baseport, --destport, --keep.
```

TCP mode is default, so no option needed.

Suggested method:
Connect to the LAB network using Ethernet! Borrow a USB network card if you dont have one.

Start your Kali VM in bridged mode, try a basic TCP flooding attack against the server provided by the instructor, or your own Debian server.

Try doing the most common attacks TCP SYN flood using hping3:

```
hping3 --flood -p 80 -S 10.0.45.12
```

You should see something like this:

```
HPING 10.0.45.12: NO FLAGS are set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
--- 10.0.45.12 hping statistic ---
352339 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

You can try different ports with TCP flooding, try port 22/tcp or HTTP(S) port 80/tcp and 443/tcp

Hints:
The tool we use can do a lot of different things, and you can control the speed. You can measure at the server being attacked or what you are sending, commonly using ifpps or such programs can help.

By changing the speed we can find out how much traffic is needed to bring down a service. This measurement can then be re-checked later and see if improvements really worked.

This allows you to use the tool to test devices and find the breaking point, which is more interesting than if you can overload, because you always can.

© license CC BY 3.0. 2026 Henrik Kramselund

```
-i --interval
       Wait  the  specified  number  of  seconds or micro seconds between sending each packet.
       --interval X set wait to X seconds, --interval uX set wait to X micro seconds.  The de-
       fault  is  to  wait one second between each packet. Using hping3 to transfer files tune
       this option is really important in order to increase transfer rate. Even  using  hping3
       to  perform  idle/spoofing  scanning  you should tune this option, see HPING3-HOWTO for
       more information.

--fast Alias for -i u10000. Hping will send 10 packets for second.

--faster
       Alias for -i u1. Faster then --fast ;) (but not as fast as your computer can send pack-
       ets due to the signal-driven design).

--flood
       Sent  packets  as fast as possible, without taking care to show incoming replies.  This
       is ways faster than to specify the -i u0 option.
```

Solution:

When your team has sent +1 million packets per second into the network, from one or two laptops - you are done.

Discussion:

Gigabit Ethernet can send up to 1.4 million packets per second, pps.

There is a presentation about DDoS protection with low level technical measures to implement at
https://github.com/kramse/security-courses/tree/master/presentations/network/introduction-ddos-testing

Receiving systems, and those en route to the service, should be checked for resources like CPU load, bandwidth, logging. Logging can also overload the logging infrastructure, so take care when configuring this in your own networks.

Exercise 21

Scapy packet crafting

- **FIXME:** reuse your Scapy exercise from presentations/pentest/simulated-ddos-workshop-exercises

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 22

Try Hydra brute force

Objective:
Try a brute force program named hydra/Xhydra

Purpose:
Learn that some protocols allow brute forcing.

Suggested method:
Log into the Unix server or use the BackTrack.

Make a short list of usernames and a short list of passwords and use hydra to brute force your way into a system. Use the editor `kate`, using `kate users.txt` and `kate pass.txt` followed by a command similar to this:

```
$ hydra -V -t 1 -L users.txt -P pass.txt 10.0.45.2 ssh
```

Hints:
When learning tools create a nice environment and check that things are working before trying to hack. So with brute forcing an account, create and test it!

Solution:
There is an FTP server with an easy to guess administrator password.

Discussion:
The hydra program can brute force a lot of different protocols and also allow a lot of tuning.

The hydra program does an online brute force attack, in some cases you can get access to data like password databases, or hash values that can be cracked in off-line brute force attacks.

© license CC BY 3.0. 2026 Henrik Kramselund

Part III

Web Attacks

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 23

⚠ Run OWASP Juice Shop 45 min



Objective:
Lets try starting the OWASP Juice Shop

Purpose:
We will be doing some web hacking where you will be the hacker. There will be an application we try to hack, designed to optimise your learning.

It is named JuiceShop which is written in JavaScript

Suggested method:
Go to https://github.com/bkimminich/juice-shop

Read the instructions for running juice-shop - docker is a simple way.

What you need

You need to have browsers and a proxy, plus a basic knowledge of HTTP.

If you could install Firefox it would be great, and we will use the free version of Burp Suite, so please make sure you can run Java and download the free version from Portswigger from:

https://portswigger.net/burp/communitydownload

Hints:
The application is very modern, very similar to real applications.

The Burp proxy is an advanced tool! Dont be scared, we will use small parts at different times.

JuiceShop can be run as a docker, and sometimes running it on Kali is the easiest learning environment.

Solution:
When you have a running Juice Shop web application in your team, then we are good.

Discussion:
It has lots of security problems which can be used for learning hacking, and thereby how to secure your applications. It is related to the OWASP.org Open Web Application Security Project which also has a lot of resources.

Sources:
`https://github.com/bkimminich/juice-shop`
`https://www.owasp.org/index.php/Category:OWASP_WebGoat_Project`

© license CC BY 3.0. 2026 Henrik Kramselund

It is recommended to buy the Pwning OWASP Juice Shop Official companion guide to the OWASP Juice Shop from https://leanpub.com/juice-shop - suggested price USD 5.99

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 24

⚠ Setup JuiceShop environment, app and proxy - up to 60min

Objective:
Run JuiceShop with Burp proxy.

Start JuiceShop and make sure it works, visit using browser. The recommended way is using Docker, see previous exercise 23

Then add a web proxy in-between. We will use Burp suite which is a commercial product, in the community edition. **This version is already installed on Kali Linux**

Purpose:
We will learn more about web applications as they are a huge part of the applications used in enterprises and on the internet. Most mobile apps are also web applications in disguise.

By inserting a web proxy we can inspect the data being sent between browsers and the application. The history of requests made and responses received is also stored, so we can go back and investigate – and document.

Suggested method:
You need to have browsers and a proxy, plus a basic knowledge of HTTP.

We will use the free version of Burp Suite, so please make sure you can run Java and download the free version plain JAR file from Portswigger from:

https://portswigger.net/burp/communitydownload

follow the Getting Started instructions at:
https://support.portswigger.net/customer/portal/articles/1816883-getting-started-with-burp-suite

The recommended setup, as it mimics the real life situation would be:

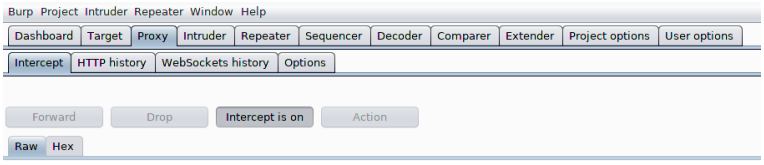
- Debian Linux running Docker with JuiceShop – say on IP `http://10.0.2.12:3000`
- Kali Linux running Burp Suite with proxy on `127.0.0.1:8080`
- Kali Linux running a browser using the proxy on `127.0.0.1:8080` to visit JuiceShop

When working you will see the Target tab showing data.

Hints:
Recommend running Burp on the default address 127.0.0.1 and port 8080.

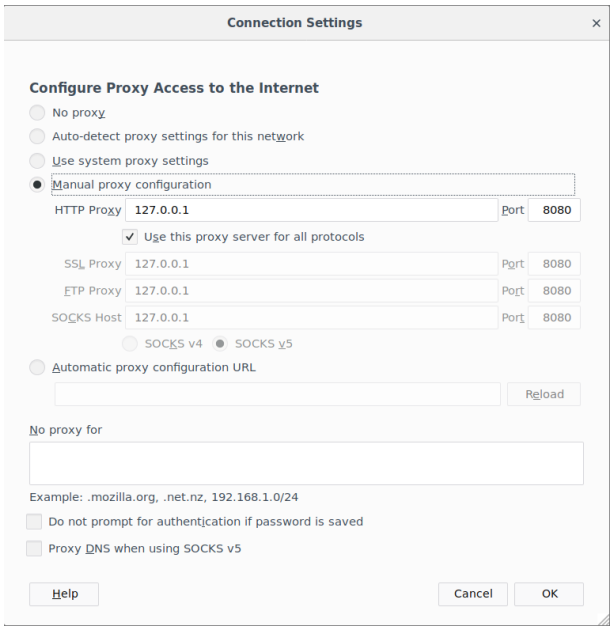
Burp Suite has a built-in Chrome browser already configured to use Burp.

Note: Burp by default has `intercept is on` in the Proxy tab, press the button to allow data to flow.



© license CC BY 3.0. 2026 Henrik Kramselund

Then setting it as proxy in Firefox:



After setting up proxy, you can visit `http://burp` and get a CA certificate that can be installed, making it easier to run against HTTPS sites.

The newest versions of Burp include a browser, making it much easier to run the tasks, pre-configured with proxy.

Solution:

When web sites and servers start popping up in the Target tab, showing the requests and responses - you are done.

Your browser will alert you when visiting TLS enabled sites, HTTPS certificates do not match, as Burp is doing a person-in-the-middle. You need to select advanced and allow this to continue.

Discussion:

Since Burp is often updated I use the plain JAR file version and a small script for starting Burp which I save in `~/bin/burp` - dont forget to add to PATH and `chmod x bin/burp+`.

```
#! /bin/sh
DIRNAME=`dirname $0`
BURP=`ls -1tra $DIRNAME/burp*.jar | tail -1`
java -jar -Xmx2g $BURP &
```

When running in production testing real sites, I typically increase the memory available using JDK / Java settings like `-Xmx16g`

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 25

⚠ JuiceShop Attacks 60min



Objective:
Hack a web application!

Try a few attacks in the JuiceShop with web proxy

> The OWASP Juice Shop is a pure web application implemented in JavaScript. In the frontend the popular AngularJS framework is used to create a so-called Single Page Application. The user interface layout is provided by Twitter's Bootstrap framework - which works nicely in combination with AngularJS. JavaScript is also used in the backend as the exclusive programming language: An Express application hosted in a Node.js server delivers the client-side code to the browser. It also provides the necessary backend functionality to the client via a RESTful API.
>
> ...
>
> The vulnerabilities found in the OWASP Juice Shop are categorized into several different classes. Most of them cover different risk or vulnerability types from well-known lists or documents, such as OWASP Top 10 or MITRE's Common Weakness Enumeration. The following table presents a mapping of the Juice Shop's categories to OWASP and CWE (without claiming to be complete).

Category Mappings

Category	OWASP	CWE
Injection	A1:2017	CWE-74
Broken Authentication	A2:2017	CWE-287, CWE-352
Forgotten Content	OTG-CONFIG-004	
Roll your own Security	A10:2017	CWE-326, CWE-601
Sensitive Data Exposure	A3:2017	CWE-200, CWE-327, CWE-328, CWE-548
XML External Entities (XXE)	A4:2017	CWE-611
Improper Input Validation	ASVS V5	CWE-20
Broken Access Control	A5:2017	CWE-22, CWE-285, CWE-639
Security Misconfiguration	A6:2017	CWE-209
Cross Site Scripting (XSS)	A7:2017	CWE-79
Insecure Deserialization	A8:2017	CWE-502
Vulnerable Components	A9:2017	
Security through Obscurity		CWE-656

© license CC BY 3.0. 2026 Henrik Kramselund

Source: Pwning OWASP Juice Shop

Purpose:
Try out some of the described web application flaws in a controlled environment. See how an attacker would be able to gather information and attack through HTTP, browser and proxies.

Suggested method:
Start the web application, start Burp or another proxy - start your browser.

Access the web application through your browser and get a feel for how it works. First step is to register your user, before you can shop.

Dont forget to use web developer tools like the JavaScript console!

Then afterwards find and try to exploit vulnerabilities, using the book from Björn and starting with some easy ones:

Suggested list of starting vulns:

- Admin Section Access the Admin Section
- Error handling Provoke and error
- Forged Feedback Post some feedback in another users name.
- Access a confidential document
- Forgotten Sales Backup Access a salesman's forgotten backup file.
- Retrieve a list of all user credentials via SQL Injection

We will use this exercise multiple times, so don't expect to solve all of these in one go.

Hints:
The complete guide Pwning OWASP Juice Shop written by Björn Kimminich is available as PDF which you can buy, or you can read it online at:
`https://pwning.owasp-juice.shop/`

Solution:
You decide for how long you want to play with JuiceShop, but try to solve at least 1-2 of these. It is OK to use the solution from the guide.

Do know that some attackers on the internet spend all their time researching, exploiting and abusing web applications.

Discussion:
The vulnerabilities contained in systems like JuiceShop mimic real ones, and do a very good job. You might not think this is possible in real applications, but there is evidence to the contrary.

Using an app like JS instead of real applications with flaws allow you to spend less on installing apps, and more on exploiting.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 26

⚠ SSL/TLS scanners 15 min

Objective:
Try the Online Qualys SSLLabs scanner https://www.ssllabs.com/ Try the command line tool sslscan checking servers - can check both HTTPS and non-HTTPS protocols!

SSLscan is available on Kali Linux and Debian

Purpose:
Learn how to efficiently check TLS settings on remote services.

Suggested method:
Run the tool against a couple of sites of your choice.

```
root@kali:~# sslscan web.kramse.dk
Version: 1.10.5-static
OpenSSL 1.0.2e-dev xx XXX xxxx

Testing SSL server web.kramse.dk on port 443
...
  SSL Certificate:
Signature Algorithm: sha256WithRSAEncryption
RSA Key Strength:    2048

Subject:  *.kramse.dk
Altnames: DNS:*.kramse.dk, DNS:kramse.dk
Issuer:   AlphaSSL CA - SHA256 - G2
```

Also run it without options to see the possibilities – and see if you can find the options to run against SMTPTLS if possible.

Hints:
Originally sslscan is from http://www.titania.co.uk but use the version on Kali, install with apt if not installed – `sudo apt install sslscan`

SMTP TLS is opportunistic encryption, if both ends support encryption it will be used. Vulnerable to active man in the middle attacks, but works well in practice.
`https://en.wikipedia.org/wiki/Opportunistic_encryption`

Solution:
When you can run and basically understand what the tool does, you are done.

Discussion:
SSLscan can check your own sites, using hostname or IP while Qualys SSLLabs only can test from hostname

Further SSLscan can be used against multiple database systems and mail systems.

Exercise 27

⚠ Internet scanners 15 min

Objective:
Try the Online scanners https://internet.nl/ and a few more.

Purpose:
Learn how to efficiently check settings on remote services.

Suggested method:
There are multiple portals and testing services which allow you to checck a domain, mail settings or web site. Others exist, feel free to suggest some.

Run tools against a couple of sites of your choice.

- `https://internet.nl/` Generic checker
- `https://www.hardenize.com/` Generic checker
- `https://www.wormly.com/test_ssl` Test TLS
- `https://observatory.mozilla.org/` Web site headers check
- `https://dnsviz.net/` DNS zone check
- `https://rpki.cloudflare.com/` Check RPKI - route validator enter IP address
  More information about this: `https://labs.ripe.net/author/nathalie_nathalie/rpki-test/`

Hints:
Especially the Mozilla Observatory is usefull for checking web site headers! I use their command line tool, https://github.com/mozilla/observatory-cli:

```
$ observatory --format report  www.kramse.dk
observatory [WARN] retrying in 1 second (attempt 1/10)
observatory [WARN] retrying in 1 second (attempt 2/10)

HTTP Observatory Report: www.kramse.dk

Score Rule                    Description
    5 content-security-policy  Content Security Policy (CSP) implemented without 'unsafe-i
    5 referrer-policy          Referrer-Policy header set to "no-referrer", "same-origin",
    5 x-frame-options          X-Frame-Options (XFO) implemented via the CSP frame-ancesto

Score: 115
Grade: A+

Full Report Url: https://observatory.mozilla.org/analyze.html?host=www.kramse.dk
```

Solution:
When you can run and understand what at least one tool does, you are done.

Discussion:
Which settings are most important, which settings are your responsebility?

Exercise 28

⚠ Nikto Web Scanner 15 min

Objective:
Try the program Nikto locally on your workstation

Purpose:
Running Nikto will allow you to analyse web servers quickly.



> **Description** Nikto is an Open Source (GPL) web server scanner which performs comprehensive tests against web servers for multiple items, including over 3200 potentially dangerous files/CGIs, versions on over 625 servers, and version specific problems on over 230 servers. Scan items and plugins are frequently updated and can be automatically updated (if desired).

Source: Nikto web server scanner http://cirt.net/nikto2

Easy to run, free and quickly reports on static URLs resulting in a interesting response

```
nikto -host 127.0.0.1 -port 8080
```

When run with port 443 will check TLS sites

Suggested method:
Run the program from your Kali Linux VM

```
Script started on Tue Nov  7 17:43:54 2006
$  nikto -host 127.0.0.1 -port 8080 ^M
---------------------------------------------------------------------------
- Nikto 1.35/1.34     -     www.cirt.net
+ Target IP:       127.0.0.1
+ Target Hostname: localhost.pentest.dk
+ Target Port:     8080
+ Start Time:      Tue Nov  7 17:43:59 2006
...
+ /examples/ - Directory indexing enabled, also default JSP examples. (GET)
+ /examples/jsp/snp/snoop.jsp - Displays information about page
retrievals, including other users. (GET)
+ /examples/servlets/index.html - Apache Tomcat default JSP pages
present. (GET)
```

Hints:
Nikto can find things like a debug.log, example files, cgi-bin directories etc.

© license CC BY 3.0. 2026 Henrik Kramselund

If the tool is not available first try: `apt-get install nikto`

Some tools will need to be checked out from Git and run or installed from source.

Solution:

When you have tried the tool and seen some data you are done.

Discussion:

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 29

⚠ Whatweb Scanner 15 min

Objective:
Try the program Whatweb locally on your workstation

Purpose:
Running Whatweb will allow you to analyse which technologies are used in a web site.

I usually save the command and the common options as a small script:

```
#! /bin/sh

whatweb -v -a 3 $*
```

Suggested method:
Run the program from your Kali Linux VM towards a site of you own choice.

```
user@KaliVM:~$ whatweb -a 3 www.zencurity.com
http://www.zencurity.com [301 Moved Permanently] HTTPServer[nginx], IP[185.129.60.
https://www.zencurity.com/ [200 OK] Email[hlk@zencurity.dk], HTML5, HTTPServer[ngi
```

Hints:
If the tool is not available first try: `apt-get install *thetool*`

Some tools will need to be checked out from Git and run or installed from source.

Solution:
When you have tried the tool and seen some data you are done.

Discussion:
How does this tool work?

It tries to fetch common files left or used by specific technologies.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 30

Real Vulnerabilities up to 30min

Objective:
Look at real vulnerabilities. Choose a few real vulnerabilities, prioritize them.

Purpose:
See that the error types described in the books - are still causing problems.

Suggested method:
We will use the 2019 Exim errors as starting examples. Download the descriptions from:

- Exim RCE CVE-2019-10149 June
  `https://www.qualys.com/2019/06/05/cve-2019-10149/return-wizard-rce-exim.txt`
- Exim RCE CVE-2019-15846 September
  `https://exim.org/static/doc/security/CVE-2019-15846.txt`

When done with these think about your own dependencies. What software do you depend on? How many vulnerabilities and CVEs are for that? Each year has critical new vulnerabilities, like the 2020 and 2021 shown here.

- CVE-2020 Netlogon Elevation of Privilege
  https://msrc.microsoft.com/update-guide/vulnerability/CVE-2020-1472
- Log4J RCE (CVE-2021-44228) - and follow up like CVE-2021-45046, also look at scanners like:
  https://github.com/fullhunt/log4j-scan

What is CVSS – Common Vulnerability Scoring System?

I depend on the OpenBSD operating system, and it has flaws too:
`https://www.openbsd.org/errata65.html`

You may depend on OpenSSH from the OpenBSD project, which has had a few problems too:
`https://www.openssh.com/security.html`

Hints:
Remote Code Execution can be caused by various things, but most often some kind of input validation failure.

Solution:
When you have identified the specific error type, is it buffer overflows? Then you are done.

Discussion:
How do you feel about running internet services. Lets discuss how we can handle running insecure code. What other methods can we use to restrict problems caused by similar vulnerabilities. A new product will often use a generic small computer and framework with security problems.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 31

⚠ PHP Passwords 30 min

```php
<?php
/**
 * We just want to hash our password using the current DEFAULT algorithm.
 * This is presently BCRYPT, and will produce a 60 character result.
 *
 * Beware that DEFAULT may change over time, so you would want to prepare
 * By allowing your storage to expand past 60 characters (255 would be good)
 */
echo "Password: henrik42 hash: ";
echo password_hash("henrik42", PASSWORD_DEFAULT);
?>
```

Objective:
Try the PHP function `password_hash` on the same password with different algorithms and compare hashes

Purpose:
Running the PHP `password_hash` function will enable you to use a quality implementation of hashing algorithms.

Suggested method:
Run the program from your Kali Linux VM, using the documentation
https://www.php.net/manual/en/function.password-hash.php

Hints:
Create a program that hash a password using multiple algorithms.

You can ask the user for input, or hard-code in the program source.

The output is expected similar to:

```
$ php my_passwordprogram.php
The password is: henrik42
The password hash using default is: $2y$10$.fwefw...2kljhfwe
... print all of the hashes possible
```

Solution:
When you have a PHP program that can hash a password you are done.

Also, explain why above example start with `$2y$10$`

Discussion:
There are multiple tunable parameters for some algorithms. Feel free to experiment with them and compare output.

Note the length of output from the hashing algoritms do NOT say how strong they are. An algorithm which output less can be more secure than one with longer output.

Password hashing is not easy, and consider an environment with 100s of applications!

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 32

Apache Benchmark 20 min

Objective:
Try the program Apache Benchmark locally on your workstation

Purpose:
Running this benchmark will allow you to analyse your webserver, perform testing and get data about performance.

On Debian this tool is in the package: `apache2-utils` (hint: `apt install apache2-utils`)

```
$
ab -n 100 https://www.kramse.org/
This is ApacheBench, Version 2.3 <$Revision: 1879490 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking www.kramse.org (be patient).....done


Server Software:        nginx
Server Hostname:        www.kramse.org
Server Port:            443
SSL/TLS Protocol:       TLSv1.2,ECDHE-RSA-AES256-GCM-SHA384,4096,256
Server Temp Key:        ECDH P-384 384 bits
TLS Server Name:        www.kramse.org

Document Path:          /
Document Length:        5954 bytes

Concurrency Level:      1
Time taken for tests:   9.596 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      651100 bytes
HTML transferred:       595400 bytes
Requests per second:    10.42 [#/sec] (mean)
Time per request:       95.962 [ms] (mean)
Time per request:       95.962 [ms] (mean, across all concurrent requests)
Transfer rate:          66.26 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:       48   71 105.5     61    1114
Processing:     6   25 157.8      9    1587
Waiting:        5    9   1.9      9      16
Total:         55   96 189.0     70    1649

Percentage of the requests served within a certain time (ms)
  50%     70
  66%     72
  75%     73
  80%     74
  90%     76
  95%     78
  98%   1124
  99%   1649
```

```
100%   1649 (longest request)
```

Suggested method:
Run the program from your Debian or Kali Linux VM

Hints:
Some tools will need to be checked out from Git and run or installed from source.

Solution:
When you have tried the tool and seen some data you are done.

Discussion:
Apache benchmark is very simple, more advanced tools can perform more advanced testing, following a login flow, ordering something, completing a purchase etc.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 33

⚠ Trying PMD static analysis 30 min

Objective:
Try the program PMD locally on your workstation

Purpose:
Running PMD will allow you to use static analysis for code.

Suggested method:
Run the program from your Debian Linux VM, this tool is free and easy to get running. It uses Java, so if you like run it on your Windows or Mac instead.

Follow instructions from the Getting Started
`https://pmd.github.io/pmd-6.43.0/pmd_userdocs_installation.html`

```
$ sudo apt install openjdk-17-jre
$ cd $HOME
$ wget https://github.com/pmd/pmd/releases/download/pmd_releases/6.43.0/pmd-bin-6.43.0.zip
$ unzip pmd-bin-6.43.0.zip
$ alias pmd="$HOME/pmd-bin-6.43.0/bin/run.sh pmd"
$
```

Note: this only creates the alias `pmd` for this session. To make this more permanent, you could add this to a profile like `.bashrc`

Next get some source code and run PMD:

```
$ git clone --branch rel/2.17.2 https://gitbox.apache.org/repos/asf/logging-log4j2.git
... downloads the source code for log4j
$ pmd -d logging-log4j2 -R rulesets/java/quickstart.xml -f text
```

Hints:
PMD uses Java, so there should be a JDK/JRE on the system, I install the one from OpenJDK above.

Solution:
When you have gotten a run of PMD going, you are done.

Discussion:
Doing the above outputs more than 4500 lines from the PMD program!

How would you proceed?

There seem to be some tedious, but easy to fix, like Unused import – importing some library which is not really used. Things like empty method, empty catch block etc. may be source missing.

First time use of a new tool will probably find a LOT.

If you are using Maven you could also use their reporting
https://maven.apache.org/plugins/maven-pmd-plugin/project-reports.html

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 34

⚠ Git hook 30 min

Objective:
Try using a Git hook locally on your workstation, to prevent something.

Purpose:
Running Git with hooks will allow you to perform actions when adding source code. For security we can prevent you from adding something to the source tree which breaks policies we agreed.

First read the documentation:
https://git-scm.com/book/en/v2/Customizing-Git-Git-Hooks

Note: today we only use client side hooks, for better security we should add hooks on the server side.

Using our existing repository:

```
user@Projects:tt$  git clone https://github.com/kramse/kramse-labs.git
Cloning into 'kramse-labs'...
user@Projects:tt$  cd kramse-labs/
user@Projects:kramse-labs$  cd .git/hooks
user@Projects:kramse-labs/.git/hooks$  cat pre-commit.sample
// Look at this file, and activate it using:
user@Projects:kramse-labs/.git/hooks$  cp pre-commit.sample pre-commit
```

Back in the repository try adding a new file with bad name:

```
user@Projects:kramse-labs$
user@Projects:kramse-labs$ touch henrikÆØÅ
user@Projects:kramse-labs$ git add henrikÆØÅ
user@Projects:kramse-labs$ git commit -m "Adding henrikÆØÅ"
Error: Attempt to add a non-ASCII file name.
This can cause problems if you want to work with people on other platforms.
To be portable it is advisable to rename the file.

If you know what you are doing you can disable this check using:
  git config hooks.allow
```

Suggested method:
Run the program Git from your Debian Linux VM

Hints:
Many hooks will depend on the language and what your policy states. Some hooks will be easy to implement, others will require others to agree. Think tabs vs spaces which will forever stay unresolved.

Solution:
When you have tried adding a file with a bad name, and gotten an error, you are done. Feel free to experiment more with the hook.

Discussion:
Many examples can be found on the internet.

Example links:

- https://blogs.vmware.com/opensource/2020/02/03/git-repo-pre-commit-hooks/ which links to the next one
- A framework for managing and maintaining multi-language pre-commit hooks.
  https://pre-commit.com/
- Blog showing how to use this framework for checking a Kubernetes file:
  https://www.magalix.com/blog/how-to-start-left-with-security-using-git-pre-commit-hooks

© license CC BY 3.0. 2026 Henrik Kramselund